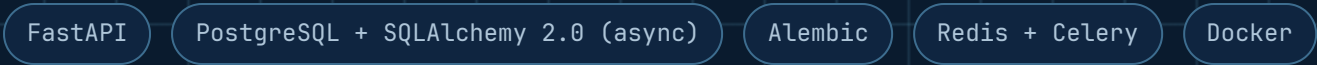


• BACKEND-ONLY SCOPE · FASTAPI · OFFICER / AUDITOR / DEVELOPER FLOW

Project & Budget Tracking System — Backend Architecture Blueprint

A complete build reference: which files go inside every module and what each one holds, how Officers initiate and staff projects, how the Auditor oversees spend and manages the workforce, and the order to build all of it in.



SHEET 00 / ROLES & FLOW

Four roles, two very different

jobs

Officer and Auditor are not the same kind of "manager." An Officer runs projects — creation, staffing, budget. The Auditor sits above all of that: money oversight across every project, plus full workforce management for developers — attendance, performance, payroll, and formal work orders.

Admin

System-level · few accounts

- Creates Departments and sets total_budget
- Creates user accounts, assigns roles
- Not involved in day-to-day project or audit work

Officer

Many accounts, one per active project cluster

- Initiates Projects under a Department
- Assigns Developers onto their Project
- Requests / allocates Project budget, capped by the Department total
- Tracks Project status through to completion

Auditor

Single oversight role, org-wide reach

- Views money utilized by every Officer, on every Project
- Manages Developers: attendance, performance, payroll
- Issues formal Work Orders to Developers
- Only role with access to the system Audit Trail

Developer

Many accounts, assigned per project

- Executes Tasks inside Projects they're assigned to
- Self-marks attendance, logs hours
- Read-only on their own Work Orders, reviews, and payslips

OFFICER FLOW

- 1 Initiate Project**
Create a Project under a Department, with a name, scope, and target dates.
- 2 Assign Developers**
Add one or more Developers onto the Project's team.
- 3 Allocate Budget**
Request a project_budget, checked against the Department's remaining total.
- 4 Monitor to Completion**
Track task status and move the Project through its lifecycle.

AUDITOR FLOW

- 1

Monitor Spend
Review every budget allocation and expense log, across all Officers and Projects.
- 2

Manage Workforce
Issue Work Orders to Developers and keep attendance records accurate.
- 3

Review Performance
Record periodic performance ratings per Developer.
- 4

Process Payroll
Approve and pay salaries, closing the loop from work to pay.

CAPABILITY	OFFICER	AUDITOR	DEVELOPER
Initiate a Project	✓	–	–
Assign Developers to a Project	✓	–	–
Allocate Project budget	✓ within dept cap	–	–
View all Budgets & Expenses org-wide	–	✓	–
Log Task hours / update status	–	–	✓ own tasks
Issue / close Work Orders	–	✓	–
View own Work Orders	–	–	✓
Mark own Attendance	–	–	✓
Review / correct any Attendance	–	✓	–
Create Performance Reviews	–	✓	–
View own Performance Review	–	–	✓ read-only
Process Salary / Payroll	–	✓	–
View own Payslip	–	–	✓ read-only

View system Audit Trail

—

✓

—

SHEET 01 / LAYERED ARCHITECTURE

Six files, one direction of dependency

Every module — Budgets, Tasks, Payroll, all of them — is built from the same set of files, each with exactly one job. A request only ever flows downward: Router → Dependency guard → Service → Repository → Model. Nothing skips a layer, and nothing calls upward.

01 models.py

≈ Eloquent Model

The SQLAlchemy class mapped to a database table — columns, foreign keys, relationships, constraints. No business logic here, only what the data looks like at rest.

```
class Project(Base): id, department_id, officer_id, name, status,
created_at...
```

02 schemas.py

≈ FormRequest + API Resource

Pydantic classes defining the shape of data crossing the API boundary — one for input (ProjectCreate), one for output (ProjectResponse). Validation happens here automatically, before your code runs.

```
AssignDeveloperRequest(developer_id: UUID) → rejects malformed IDs
with 422
```

03 repository.py ≈ Eloquent query scopes, made explicit

The only layer allowed to touch the DB session directly. Exposes plain methods like `get_by_id` or `list_by_officer` and hides every line of SQLAlchemy query syntax from the rest of the app.

```
project_repository.add_member(project_id, developer_id)
```

04 service.py ≈ Action / Service class

Where business rules live: the budget cap check, ownership checks, the overtime flag, payroll math. Calls one or more repositories, never touches SQL directly, and is fully testable without HTTP.

```
if requested_amount + allocated > department.total_budget: raise
BudgetExceeded()
```

05 router.py ≈ Route + Controller method

The thin HTTP layer. Declares the path and method, injects auth and guards, hands the validated schema to the service, returns the response schema. No logic of its own.

```
@router.post("/projects/{id}/assign-developer") → calls
project_service.assign_developer()
```

+ dependencies.py ≈ Middleware / Policy

Optional per-module file for role guards that are specific to that module — kept separate from the generic `require_role()` in Core so ownership checks stay close to the domain they protect.

```
require_officer_owns_project(project_id, current_user) – used only in
projects/router.py
```

Sixteen modules, grouped by who actually uses them

Each module is a folder under `app/modules/<name>/`. The manifest under each card is the literal file list to create and what belongs in it — this is what "which files, what to store" means in practice.

MOD-01

AUDITOR

ADMIN

Departments

The budget container every Project draws from.
Set up by Admin, watched by Auditor.

FILES

<code>models.py</code>	<code>Department(id, name, total_budget, created_at)</code>
<code>schemas.py</code>	<code>DepartmentCreate</code> · <code>DepartmentResponse</code> · <code>BudgetSummary</code>
<code>repository.py</code>	<code>get_by_id()</code> · <code>sum_allocated_by_department()</code>
<code>service.py</code>	<code>get_budget_summary()</code> → total vs allocated vs remaining
<code>router.py</code>	<code>POST / (Admin)</code> · <code>GET /{id}/budget-summary (Auditor)</code>

MOD-02

OFFICER

AUDITOR

Budgets

Officer requests allocation for their Project;
Auditor audits every allocation and expense logged against it.

FILES

<code>models.py</code>	<code>BudgetAllocation(id, project_id, amount, allocated_by_id, allocated_at)</code> · <code>ExpenseLog(id, project_id, amount, category, note, logged_by_id)</code>
<code>schemas.py</code>	<code>BudgetAllocateRequest</code> · <code>ExpenseLogCreate</code> · <code>BudgetAuditView</code>
<code>repository.py</code>	<code>sum_allocated_for_department()</code> · <code>list_expenses_by_project()</code> · <code>list_all()</code>
<code>service.py</code>	<code>allocate_to_project()</code> · <code>log_expense()</code>
<code>router.py</code>	<code>POST /allocate (Officer)</code> · <code>POST /expenses (Officer)</code> · <code>GET /audit (Auditor)</code>

KEY RULE

`allocate_to_project()` rejects if `sum(project budgets in department) + amount > department.total_budget`

MOD-03

OFFICER

Projects

Officer initiates the Project and staffs it — this module owns both the project record and its developer roster.

FILES

<code>models.py</code>	<code>Project(id, department_id, officer_id, name, status, created_at)</code> · <code>ProjectMember(id, project_id, developer_id, assigned_by_id, assigned_at)</code>
<code>schemas.py</code>	<code>ProjectCreate</code> · <code>ProjectResponse</code> · <code>AssignDeveloperRequest</code> · <code>StatusUpdate</code>
<code>repository.py</code>	<code>get_by_id()</code> · <code>list_by_officer()</code> · <code>add_member()</code> · <code>list_members()</code>
<code>service.py</code>	<code>create_project()</code> → calls <code>budget_service.allocate_to_project()</code> · <code>assign_developer()</code> · <code>advance_status()</code>
<code>router.py</code>	<code>POST /</code> · <code>POST /{id}/assign-developer</code> · <code>PATCH /{id}/status</code>
<code>dependencies.py</code>	<code>require_officer_owns_project(project_id, user)</code>

KEY RULE

Only the `officer_id` who owns the project (or Admin) can assign developers or edit its budget

Tasks

Where Developers do the actual work, scoped to Projects the Officer already staffed them onto.

FILES

<code>models.py</code>	<code>Task(id, project_id, assigned_to_id, title, status, estimated_hours, logged_hours)</code>
<code>schemas.py</code>	<code>TaskCreate · TaskUpdate · TaskResponse</code>
<code>repository.py</code>	<code>get_by_id() · list_by_project() · increment_logged_hours()</code>
<code>service.py</code>	<code>log_progress() → flags</code> <code>"Overtime Risk" if logged > estimated</code>
<code>router.py</code>	<code>PATCH /{id}/progress · GET /? assigned_to=me</code>

KEY RULE

A Developer can only transition tasks where `assigned_to_id = current_user.id`

MOD-05

AUDITOR

Work Orders

A formal directive the Auditor issues to a Developer — separate from day-to-day Tasks, used as the official record of an assignment.

FILES

<code>models.py</code>	<code>WorkOrder(id, developer_id, issued_by_id, project_id?, title, description, status, issued_at, closed_at)</code>
<code>schemas.py</code>	<code>WorkOrderCreate</code> · <code>WorkOrderResponse</code>
<code>repository.py</code>	<code>list_by_developer()</code> · <code>list_open()</code>
<code>service.py</code>	<code>issue_work_order()</code> · <code>close_work_order()</code>
<code>router.py</code>	<code>POST / (Auditor)</code> · <code>GET /mine (Developer)</code> · <code>GET / (Auditor, all)</code>

KEY RULE

Only the Auditor role can issue or close a work order

MOD-06

AUDITOR

DEVELOPER

Attendance

Daily presence per Developer — self check-in/out, with the Auditor able to review or correct any record.

FILES

<code>models.py</code>	<code>AttendanceRecord(id, developer_id, date, check_in, check_out, status, marked_by_id)</code>
<code>schemas.py</code>	<code>CheckInRequest</code> · <code>CheckOutRequest</code> · <code>AttendanceResponse</code>
<code>repository.py</code>	<code>get_for_date()</code> · <code>list_by_developer_range()</code>
<code>service.py</code>	<code>check_in()</code> · <code>check_out()</code> · <code>mark_leave()</code> (Auditor override)
<code>router.py</code>	<code>POST /check-in (self)</code> · <code>GET /me (self)</code> · <code>GET /{developer_id} (Auditor)</code>

KEY RULE

A Developer can only check in/out for themselves; Auditor can view and correct any record

MOD-07

AUDITOR

DEVELOPER

Performance Reviews

Periodic ratings the Auditor records per Developer, informed by task completion data plus written notes.

FILES

<code>models.py</code>	<code>PerformanceReview(id, developer_id, reviewed_by_id, period, rating, notes, created_at)</code>
<code>schemas.py</code>	<code>ReviewCreate</code> · <code>ReviewResponse</code>
<code>repository.py</code>	<code>list_by_developer()</code> · <code>latest_for_developer()</code>
<code>service.py</code>	<code>create_review()</code> — can pull task/hour stats from Tasks as a reference
<code>router.py</code>	<code>POST / (Auditor)</code> · <code>GET /{developer_id} (Auditor)</code> · <code>GET /me (Developer)</code>

KEY RULE

A Developer can only read their own reviews, never create or edit one

Payroll & Salary

Auditor processes and records salary payments — base pay, bonus, deductions, net pay per period.

FILES

<code>models.py</code>	<code>SalaryRecord(id, developer_id, period, base_salary, bonus, deductions, net_pay, status, processed_by_id, paid_at)</code>
<code>schemas.py</code>	<code>SalaryCreate</code> · <code>PayslipView</code>
<code>repository.py</code>	<code>list_by_developer()</code> · <code>list_by_period()</code>
<code>service.py</code>	<code>process_salary()</code> → computes <code>net_pay</code> · <code>mark_paid()</code>
<code>router.py</code>	<code>POST / (Auditor)</code> · <code>GET /{developer_id} (Auditor)</code> · <code>GET /me (Developer)</code>

KEY RULE

Only Auditor can create or mark a salary record paid; Developer has read-only access to their own payslips

Audit Trail

Immutable, append-only log of every change to sensitive records — the technical backbone behind the Auditor's oversight.

FILES

<code>models.py</code>	<code>AuditEntry(id, entity_type, entity_id, action, performed_by_id, before, after, timestamp)</code>
<code>schemas.py</code>	<code>AuditEntryResponse</code>
<code>repository.py</code>	<code>list_by_entity()</code> · <code>list_by_actor()</code>
<code>service.py</code>	<code>record()</code> — called internally by other services, never by a router
<code>router.py</code>	<code>GET / (Auditor)</code> · <code>GET /?entity=budget&id=</code>

KEY RULE

Written automatically via SQLAlchemy event listeners on Budget, Salary, and role changes — never called directly by a router

Reporting & Analytics

Aggregated views for both actors — Officer-facing project summaries, Auditor-facing org-wide spend and workforce reports.

FILES

<code>schemas.py</code>	<code>DashboardSummary</code> · <code>ExportRequest</code>
<code>repository.py</code>	aggregate queries — spend vs allocated, hours vs estimate, attendance rate
<code>service.py</code>	<code>build_officer_dashboard()</code> · <code>build_auditor_dashboard()</code> · <code>export_pdf()</code>
<code>router.py</code>	<code>GET /dashboard</code> (scoped by role) · <code>GET /export</code>

KEY RULE

Same endpoint, different data — response is scoped server-side by `current_user.role`; an Officer never receives cross-department figures

MOD-11

SHARED

Auth & Identity

Issues and verifies who's making a request. Every other module depends on this existing first.

FILES

schemas.py	LoginRequest · TokenPair · RefreshRequest
service.py	authenticate() · issue_tokens() · revoke()
router.py	POST /login · POST /refresh · POST /logout
security.py	hash_password() · verify_password() · create_jwt()

MOD-12

SHARED

Users & RBAC

Owns user records and the role checks every other module relies on.

FILES

models.py	User(id, name, email, hashed_password, role, department_id, is_active)
schemas.py	UserCreate · UserResponse · RoleEnum(Admin, Officer, Auditor, Developer)
repository.py	get_by_email() · list_by_role()
service.py	create_user() · change_role()
router.py	GET /me · POST / (Admin) · GET /?role=
dependencies.py	require_role(*roles) → require_officer, require_auditor, require_developer

MOD-13

SHARED

Notifications

Tells people things happened, without making them wait for it.

FILES

models.py	Notification(id, user_id, message, read, created_at)
tasks.py	Celery task: send_notification() — fired on assignment, salary paid, budget allocated
router.py	GET /unread · PATCH /{id}/read

MOD-14

SHARED

File Storage

Handles anything that isn't a database row — contracts, receipts, work order attachments.

FILES

models.py	FileAsset(id, owner_id, entity_type, entity_id, s3_key, uploaded_at)
service.py	generate_presigned_url() · confirm_upload()
router.py	POST /presign · GET /{id}

MOD-15

SHARED

Realtime Gateway

The one module that keeps a connection open instead of answering a single request.

FILES

connection_manager.py	tracks active WebSocket connections per user
router.py	WS /ws/projects/{id} — pushes task-board and notification events

MOD-16

SHARED

Core / Shared

Not a module — the plumbing every other module imports from.

FILES

config.py	pydantic-settings, loaded once from environment
database.py	async engine + session factory + get_db() dependency
exceptions.py	BudgetExceeded, NotOwner, etc. → global handlers, one JSON error shape
base_schema.py	shared Pydantic base classes, pagination envelope

What separates a demo from something you’d trust in production

Grouped by the problem each one solves — not extra features, but what an industry-level backend is expected to have before real budget and payroll data goes through it.

■ DATA & PERSISTENCE PostgreSQLprimary relational store SQLAlchemy 2.0 (async)ORM + async session Alembicversioned schema migrations Rediscache + Celery broker + pub/sub	■ SECURITY JWT access + refreshshort-lived sessions Argon2 hashingpassword storage slowapi rate limitingabuse / brute-force protection Vault / Dopplersecrets, out of .env in prod	■ ASYNC & BACKGROUND WORK Celery + Redispayroll runs, reports, emails FastAPI BackgroundTasksfire-and-forget, lightweight Celery beatscheduled jobs — monthly payroll, reports
■ TESTING & QUALITY pytest + pytest-asynciounit + integration tests httpx AsyncClientAPI-level test client factory_boytest data fixtures per role coverage.pyenforced threshold in CI	■ OBSERVABILITY structlogstructured, queryable logs Sentryerror tracking + alerting Prometheus + Grafanametrics + dashboards OpenTelemetrydistributed tracing	■ DEVOPS & DELIVERY Docker + docker-composereproducible local + prod env GitHub ActionsCI: lint, test, build on push Gunicorn + Uvicorn workersproduction ASGI serving Nginreverse proxy + TLS termination

■ API DESIGN

/api/v1 versioning safe to evolve later

Consistent error envelope one shape for every failure

Pagination + filtering standard applied the same way everywhere

Swagger / ReDoc auto-generated, always in sync

■ STORAGE & FILES

S3 / MinIO contracts, receipts, attachments

Presigned URLs API server never streams file bytes

SHEET 04 / BUILD ROADMAP

Six phases, in the order that actually works

Each phase only becomes buildable once the one before it exists. Auth has to exist before roles mean anything; Officer/Projects has to exist before the Auditor has any money or people to oversee.

00

Foundation

Skeleton & environment IN PROGRESS

Project structure, pydantic-settings config, async DB session, Alembic initialized, health-check route. This is where you are right now while working through the layered-pattern guide.

app/ layout

Postgres connection

Alembic init

.env + settings

01

Identity

Auth, Users, RBAC — four roles

Build Admin, Officer, Auditor, Developer as a single role enum with a shared `require_role()` guard. Everything downstream imports this.

JWT login/refresh

Password hashing

`require_officer` / `require_auditor`

02

Officer Core

Departments, Budgets, Projects, Tasks

Departments first, then the budget allocation cap, then Projects with developer assignment, then Tasks with hour-logging and ownership checks.

Budget cap logic

`assign_developer()`

Overtime flag

03

Auditor Core

Work Orders, Attendance, Performance, Payroll

The workforce-management suite. Attendance before Performance, Performance before Payroll — each one's data feeds the next.

Attendance check-in/out

Performance reviews

Payroll processing

04

Insight

Audit Trail, Reporting, Notifications, Realtime

Wire up the event listeners for the Audit Trail, build the two dashboard variants, and add live task-board updates over WebSockets.

SQLAlchemy event hooks

Role-scoped dashboards

WS gateway

05

Hardening

Tests, observability, containerization

The phase most side projects skip — and exactly the one that makes this "industry-level" instead of "portfolio project."

pytest suite + CI

Sentry + structlog

Docker + Nginx

Rate limiting

